

Release Summary

VeriFi

Team Members:

Christopher Zarraga (PO), Sai Gonuguntla, Srushti Chaudhari, Ethan Williams, Divya Machiraju

Date: July 21, 2026

1. Key User Stories and Acceptance Criteria

US-1.3 and US-2.1 — Financial document ingestion

As a system administrator, I want Fidelity policy documents converted into validated, embedded document chunks so that the vector store can retrieve reliable source material for user questions.

Acceptance criteria:

- The ingestion pipeline reads one or more PDF documents using PyMuPDF
- Extracted text is cleaned and normalized before chunking
- Documents are split into word aware chunks with overlap to preserve context
- Each chunk gets a generated embedding
- Each exported record contains:
 - chunk index
 - chunk text
 - source document
 - page number
 - embedding
- Records are exported as JSONL for loading by the C++ vector store.
- Validation rejects empty chunks, missing metadata, missing embeddings, and inconsistent embedding dimensions.
- The vector store can load the generated JSONL without manual reformatting.

US-2.3 and US-3.4 : Grounded RAG answers with citations

As a Fidelity user, I want responses generated using retrieved documents and include document citations so that I can trust the information provided.

Acceptance criteria:

- The system retrieves relevant policy document chunks for a user question.
- LLM answer is generated only from retrieved context via the grounding system prompt.
- Response includes source citations (document name, page number, supporting text/score when available).

- Answers do not invent financial policy details that are not present in retrieved chunks.

US-3.1 : Citations visible in the chat UI

As a Fidelity user, I want to see which documents support the answer to my question.

Acceptance criteria:

- Frontend displays source citations with each assistant response.
- Frontend and backend request/response contracts are integrated end-to-end.
- Relevant document chunks and citations appear with the answer.

US-4.1 : Accurate customer-service style experience

As a Fidelity account holder, I want a customer service experience that feels real and accurate.

Acceptance criteria:

- Users can ask a natural language policy question in the React chat UI.
- The system returns a coherent, conversational answer grounded in retrieved Fidelity policy documents.
- LLM integration uses a prompt pipeline “generate_rag_response” with retrieved chunks.
- QA accuracy checks pass for the release test.

US-4.2 — Chat sources

As a user of VeriFi, I want to see documents and questions used in responses over the course of the chat.

Acceptance criteria:

- Users can view chat documents in the interface.
- Past responses retain associated source/passage information where implemented.

US-2.4 — Testing plan for VeriFi

As a developer, I want a testing plan for VeriFi so that the system can be evaluated consistently as new features are added.

Acceptance criteria:

- Documented QA checklist exists prompts/evaluation/qa_checklist.md
- Automated QA tests exist for prompt/RAG response validation prompts/evaluation/run_qa_tests.py.

- Test questions cover basic policy, fees, transfers, beneficiaries, insufficient context, prompt injection, conflicting sources, empty input, unrelated questions, and multi-source questions.

2. Known Problems

- Automated QA suite currently uses sample retrieved chunks for several cases; end-to-end accuracy still depends on the quality of real vector-store retrieval for live demos.
- Hallucination risk is reduced by RAG + grounding prompts + citations, but not fully eliminated for every edge-case policy question.
- Prompt-injection resistance is tested in QA-006; broader adversarial document corpora were not exhaustively tested.

3. Product Backlog

- Expand live end-to-end retrieval tests against the full Fidelity document corpus (not only sample chunks).
- Add stronger hallucination regression tests comparing answers against gold citations from official pages.
- Improve handling of conflicting sources with explicit UI indicators when documents disagree.
- User document upload / personalized corpus support.